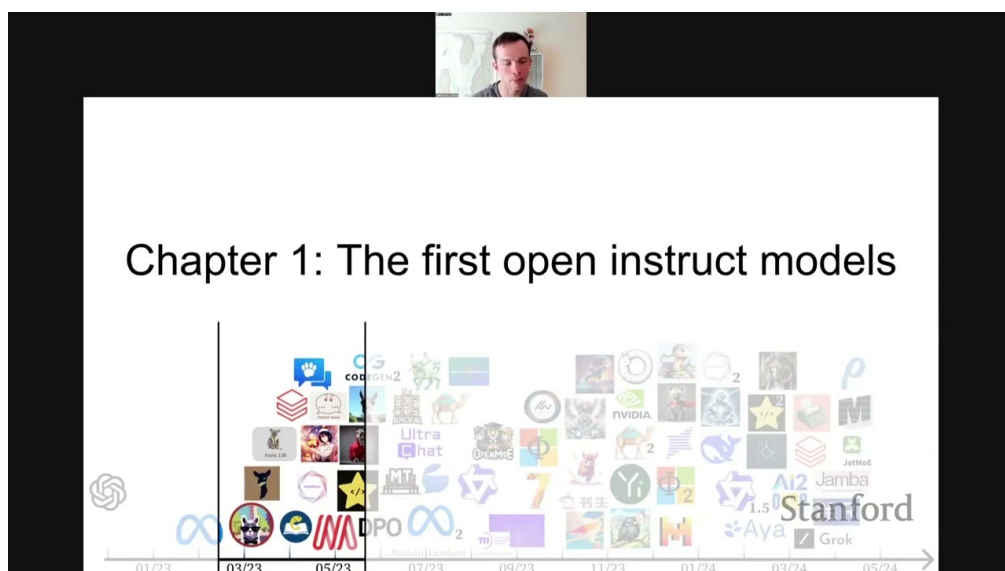


访谈笔记

[CS25] Shaping the Future of AI from the History of Transformer — Hyung Won Chung, OpenAI

基于 Stanford CS25 课程资料整理

2024 年 5 月 15 日



视频作者/频道: Stanford CS25: Transformer 深度研讨

发布日期: 2024 年 5 月 15 日

视频时长: 1h05m

视频链接: <https://cs.stanford.edu/courses/cs25-2024>

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 演讲背景与议程	2
1.1 演讲人背景	2
1.2 研究动机与议程	2
1.3 方法论与组织	3
1.4 本章小结	3
2 掌握变革：驱动因素与“苦涩教训”	4
2.1 识别多重驱动力	4
2.2 Compute 与 Scaling Laws	4
2.3 苦涩教训的反思	5
2.4 Compute vs Structure checklist	5
2.5 本章小结	6
3 Transformer 架构演化的结构解码	7
3.1 变体速览	7
3.2 参数共享与 attention 重构	7
3.3 跨层共享实验的观察	8
3.4 信息流与 cross attention 瓶颈	9
3.5 本章小结	9
4 底层表示与序列建模	10
4.1 Embedding 与流程视图	10
4.2 Tokenization 与位置信息	10
4.3 本章小结	11
5 案例解读与工程支撑	12
5.1 机器翻译与学术 finetune 数据	12
5.2 多轮对话与 attention 模式	12
5.3 工程 checklist	13
5.4 指令微调数据的长度分布	13
5.5 本章小结	14
6 未来轨迹与可撤销结构	15
6.1 可撤销结构的哲学	15
6.2 未来预测与工程弹性	15
6.3 工程要点清单	15
6.4 本章小结	16
7 观察与行动	17
7.1 关键幻灯片速读	17
7.2 实践反馈循环	17
7.3 本章小结	17

8 总结与延伸 18

8.1 核心总结表 18

8.2 进一步阅读 18

8.3 本章小结 18

1 演讲背景与议程

1.1 演讲人背景

Hyung Won Chung 是 OpenAI 的研究科学家，从 Google Brain 到 OpenAI，他始终聚焦大语言模型的高质量 instruction tuning 与可扩展训练流程。从 Flan-T5 到 PaLM 的指令微调管线，他不仅推动大型模型落地，更利用工程经验折射出理论洞察。

讲者背景速览

Hyung 参与了 PaLM-Instruct、T5X 训练框架、Scaling Instruction-Finetuned Language Models 等工作。他将 Transformer 从工程日报中抽象成“可撤销结构 + 规模”的通用思维，强调历史中留下的设计线索如何预示未来。

1.2 研究动机与议程

“AI is advancing so fast that it is hard to keep up / But not enough attention goes to the old things.” 这一开场将我们引向历史透视的逻辑：先识别驱动力，再回顾关键结构，最后以观察残差预测走势。第一部分的幻灯片（Slide 002-004）把焦点放在“研究者正在被短期技巧牵引”与“我们需要主动理解变革”的核心对话。

What does it mean to study the change itself?

- 1 Identify the dominant driving forces behind the change
- 2 Understand the dominant driving forces
- 3 Predict the future trajectory

3

图 1: Slide 2 强调“追新容易，回头看旧结构更难”作为研究的基本张力。

三步法简述

1. 识别 dominant driving force（如算力增长）；
2. 理解历史结构如何为当下提供可撤销的 inductive bias；
3. 用残差观察判断未来 trajectory，并保留“回退门槛”。

Toy experiment: dropping a pen

- 1 Identify the dominant driving forces: gravity
- 2 Understand gravity: Newtonian mechanics provides a good model
- 3 Predict the future trajectory of the pen $y(t) = \frac{1}{2}gt^2$

4

图 2: Slide 3 将“识别-理解-预测”步法形象化，强调分析的顺序。

三步法的时间维度

分析不是一条平行线，而是一条由近及远的时间轴。Slide 3 强调：先搞清楚今天的 driving force，再回到过去的结构，最后预测未来的 trajectory，避免用未来的不确定性干扰现实的 measurement。

1.3 方法论与组织

Hyung 用一个“掉笔”实验比喻未来预测的困难：需要抓住布局、理解引力、再推理轨迹。因此本讲的逻辑层层递进，先从计算与规模谈起，再拆解 Transformer 结构，随后结合真实案例，最后回到未来战略与工程实践。

构建信念的三层路径

Find the driving force → Decode the structure → Predict how to scale while keeping options reversible。

演讲的教学逻辑

Hyung 以工程师的视角把 Transformer 史分为：driving force 维度、architecture 维度、deployment 维度。每一章都以“教学逻辑”组织而非字幕流水，以便读者理解为何要关注某张幻灯片而非逐字复述。

1.4 本章小结

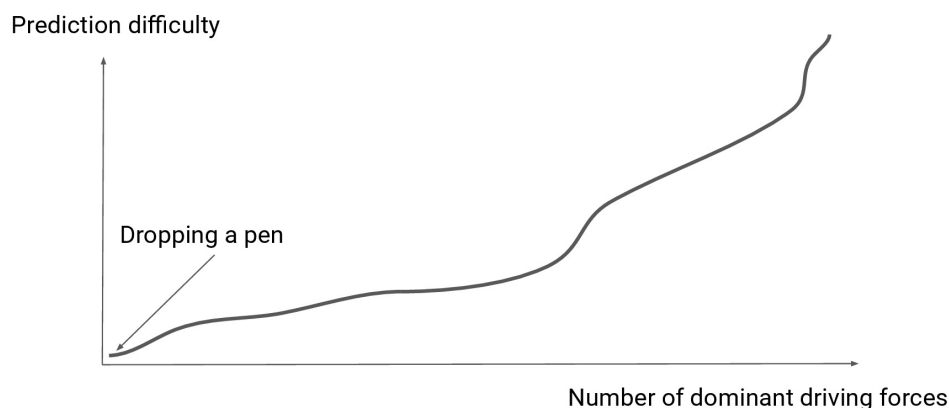
本章建立了讲座采用的三步视角：先洞察动因，再还原结构，最后评估未来走向，并确定本笔记的写作顺序。

2 掌握变革：驱动因素与“苦涩教训”

2.1 识别多重驱动力

Hyung 用“掉笔”实验说明：真实世界充斥多种 driving force，因此直觉上的视野往往过窄。幻灯片 004-005 的可视化把 pen drop 的力、空气阻力、重力并列，并标注“难点在于力的交互与参数数量”，进而把焦点拉向 AI 研究中的 dominant driver。

Predicting the future trajectory is difficult because there are many driving forces and the complexity of their interactions



5

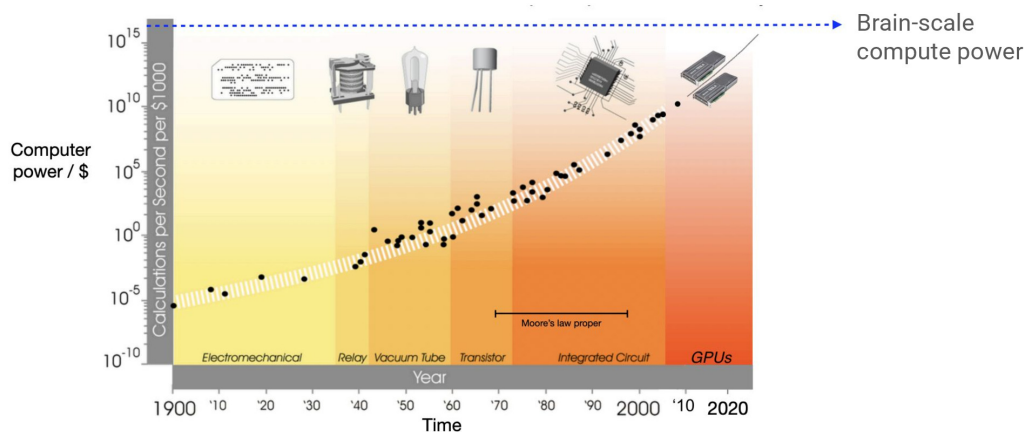
图 3: Slide 4-5 展示 pen drop 实验，帮助解释为什么预测未来时需要把多个 driving forces 组合起来。

现实世界的 driving forces

- 重力、空气阻力：暗示模型训练中不同 inductive bias 的协同；
- 复杂交互：提示我们不能只看某一项指标；
- Dominant driver 是 compute & scale。

2.2 Compute 与 Scaling Laws

幻灯片 006-009 把“brain-scale compute power”具体化：每 5 年算力增长约 10 倍，趋势来自 Rich Sutton 的 WAIC keynote。Hyung 特别强调，工程选型需要与这条趋势对齐，短期的降维或过度结构可能在未来 10 倍算力释放下失去价值。



Roughly, 10x more compute every 5 years

Figure from Rich Sutton's [WAIC keynote](#)⁷

图 4: Slide 6 的 SSE 轨迹图说明了算力与 AI 进展的强相关性。

Scaling law 的工程启示

- Compute growth 超越常规模型能力；
- 更弱的 inductive bias 让模型能在更多任务中泛化；
- 未来 10 倍算力释放意味着需要 reserve architecture 选项。

“Less structure” 的耐久性

The Bitter Lesson 告诉我们：过去 70 年进展的主引擎不是精致结构，而是 general methods 与规模优化。Hyung 以此警示：即便现在弱结构略逊，长期依赖 general method 才能迎接指数级算力。

2.3 苦涩教训的反思

与其在每个任务上加入 ad-hoc bias，不如保持结构可撤销。幻灯片 008-012 通过“更少 structure、添加更多 data”对比强化这个观点：先选择 less structure、等 scale up 再“undo”。这不仅是一种训练策略，更是一种组织研究的思维。

不要过早锁死结构

过度依赖 task-specific head 或预定义 pipeline，使得未来 scale 无法快速撤销这些 inductive bias；结果是在更多算力到来时继续被旧结构牵引。

2.4 Compute vs Structure checklist

Hyung 在 Slide 007 与 Slide 009 之间反复提醒我们：需要在“compute”和“structure”之间保持动态平衡。以下 checklist 是在做 architecture decision 时可以复盘的点。

维度	行动要点
Compute ramp-up	确保候选结构可以在更高算力下训练而不引入新的 instability;
Inductive bias	记下每个 bias 的初衷与条件, 在扩张后审视是否仍带来 signal;
Deployment margin	把 latency/throughput 作为最高优先级, 避免只看 loss。

别让参数增加掩盖 scale risk

如果只关注 loss 降低, 很容易忽略模型对特定 bias 的依赖; 在 scale-up 过程中持续监测 structure cost 才能防止未来被旧架构锁死。

2.5 本章小结

本章聚焦 dominant driving force, 与 scaling law、Bitter Lesson 同步: 强调“先对齐 compute, 再决定结构”, 而后者只在需要时再逐步引入。

3 Transformer 架构演化的结构解码

3.1 变体速览

Transformer 的三大变体 (Slide 016) 分别是 encoder-decoder、encoder-only 与 decoder-only。从 attention pattern 到输出接口，每个变体对应一组 tradeoff: encoder-decoder 多了 cross attention, encoder-only 更适合分类任务, decoder-only 在生成任务中参数共享省时。

Transformer architectures variants

1. Encoder-decoder
2. Encoder-only
3. Decoder-only (least structure)

17

图 5: Slide 16 视觉化三种 Transformer 架构差异，便于理解结构选择的方向。

三类架构的对比

- Encoder-decoder: cross attention + 分离参数，适合翻译与长输入；
- Encoder-only: 双向 self-attention, 通常用于理解任务；
- Decoder-only: 参数共享 + 递归 causal, 适合纯生成与指令微调。

3.2 参数共享与 attention 重构

Slides 038-051 展示如何逐步从 encoder-decoder 过渡到 decoder-only: 1) 共享 cross/self attention; 2) 共享 encoder/decoder 参数; 3) decoder layer 与 encoder layer 一一对应; 4) encoder 自注意力变为 causal。每一步都是在“保留必要模式”的前提下移除冗余结构。

Summary of the differences

	Encoder-decoder	Decoder-only
Additional cross attention		
Parameter sharing		
Target-to-input attention pattern		
Input attention		

39

图 6: Slide 38 对比 encoder-decoder 与 decoder-only 的 attention & parameter flow。

步骤	架构重构要点	工程意涵
共享 attention 参数	cross/self 统一，减小内存开销	训练与推理使用同一计算图
共享 encoder/decoder 参数	简化优化目标	微调 pipeline 更通用
层内对齐注意力	decoder layer l 只 attends encoder layer l	降低跨层通信带来的复杂度
Encoder causal 化	使 encoder 与 decoder 同步 causal pattern	赋予 decoder-only 生成能力

结构演化的设计原则

每次重构都要保留“会后可撤回”的能力：先去除结构，再观察 scale 反应，最终在未来需要时再引入新的 inductive bias。

3.3 跨层共享实验的观察

Slides 041-047 进一步对比了 encoder-decoder 与 decoder-only 的每一层 attention pattern：包括 cross attention 与 self-attention 共享、不同层之间的参数耦合以及 causal 迈向。讲者用这一系列幻灯片强调：每次共享都要判定 performance impact 与 training stability，然后再决定是否进一步撤销或恢复该结构。

Summary of the differences

	Encoder-decoder	Decoder-only
Additional cross attention	Separate cross attention	Self-attention serving both roles
Parameter sharing		
Target-to-input attention pattern		
Input attention		

42

图 7: Slide 41 总结当前架构演化的四个关键动作，为工程实验提供 checklist。

共享实验的决策要点

1. 共享 attention 参数：观察 training curve 与 memory usage；
2. encoder 与 decoder 参数一体化：watch for optimization interference；
3. decoder 绑定 encoder 同层：验证 cross-layer gradient flow；
4. encoder 变 causal：确认 autoregressive generation 不再需要 cross attention。

3.4 信息流与 cross attention 瓶颈

Slides 059-062 提出一个关键问题：如果 decoder 只 attend encoder 最后一层激活，那么 deep encoder 的信息会在到达 decoder 前失去分层语义。Hyung 通过 vision 类比说明底层与顶层 encode 的差异，暗示需要融合多层信息或局部 attention。

Cross attention 信息瓶颈警示

当 encoder 非常深时，仅访问最终激活会漏掉低层局部信息；建议探索 cross-layer attention 或 skip connection 解决。

3.5 本章小结

本章拆解 Transformer 架构的演化路径，既介绍变体差异，也强调每一步重构的原则与潜在瓶颈，为后续部署与未来趋势打下基础。

4 底层表示与序列建模

4.1 Embedding 与流程视图

Slides 017-021 提供了 Transformer 底层表示的多视角: tokenization、embedding、sequence model 的处理流程。右侧的概率分布/向量弹簧图 (Slide 018) 显示了如何将 tokens 映射为 $[d_{\text{model}}, \text{length}]$ 的矩阵, 再交由自注意力模块捕捉序列关系。



19

图 8: Slide 18 说明了 tokenization 到 embedding 的 transformation, 强调 shape change。

序列建模的梯度流程

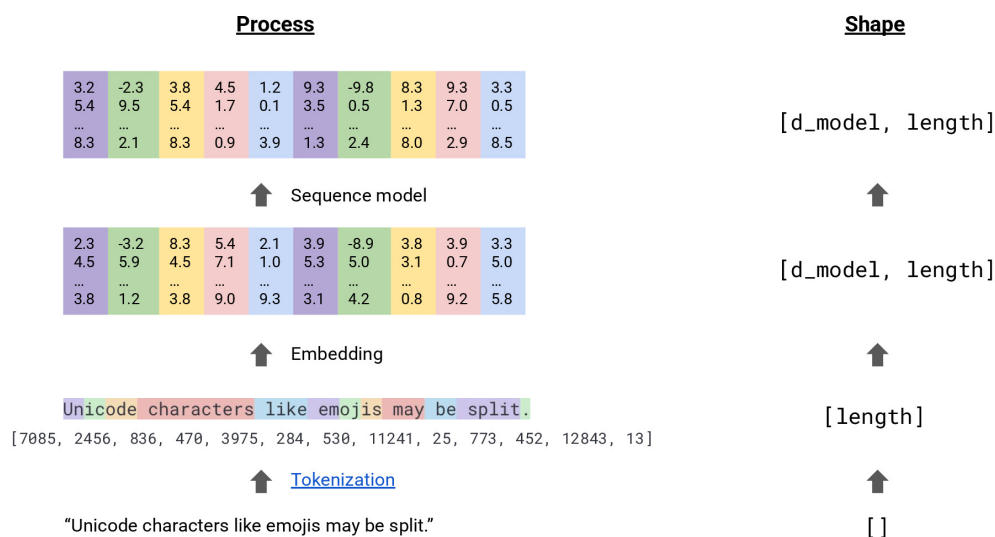
1. Tokenization 产生 $[\text{length}]$ 的索引序列;
2. Embedding 将索引映射为高维 $[d_{\text{model}}, \text{length}]$;
3. Self-attention/MLP 迭代捕捉位置与语义;
4. 输出再次映射回 vocabulary space。

4.2 Tokenization 与位置信息

Slides 019-022 进一步拆解 tokenization 过程: Unicode (如 emoji) 可能被拆分为多个 token, 而输入序列则通过位置编码与 embedding 一起喂入 Transformer。讲者强调: 不要让 tokenization 过度依赖任务 (如仅对话), 否则模型将失去 generality。

Tokenization 的普适原则

保持 pipeline 可扩展: 确保 tokenizer 能处理新字符、保持 position embedding 的连续性, 并避免在 early stage 针对某些 tokens 做特殊处理, 否则未来 scale-up 时难以撤销。



21

图 9: Slide 20 细化 tokenization 后的 process 以及多维向量 ([d_model, length]) 的分布。

不要把 position encoding 固化到 tokenizer

将 position encoding 嵌入 tokenizer 或在 preprocessing 里硬编码，会在以后很难升级到异长或者 streaming 模式；建议把 position strategy 与 tokenization 分离。

4.3 本章小结

本章通过底层表示流程与 tokenization 的视图，提醒我们即便在高层结构演化后，最底层的输入表示也必须保持 general，用以承接未来的可撤销架构。

5 案例解读与工程支撑

5.1 机器翻译与学术 finetune 数据

在 Machine Translation 以及 Scaling Instruction-Finetuned Language Models (Slides 53-58) 中, encoder-decoder 曾经获益良多: 输入与输出分别在不同语言、不同长度段。随着数据转向多轮对话与 open-ended generation, 这种差异缩小, decoder-only 开始胜出。

Example 1: machine translation

When Transformer was introduced in 2017, translation was a popular and difficult task

Input and target are in different languages

If the only goal is to learn translation, separate parameters make sense

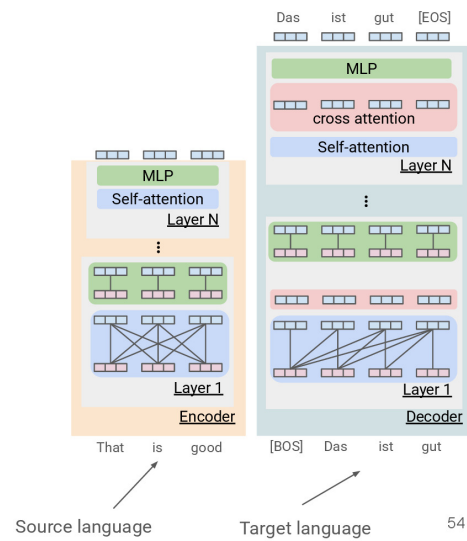


图 10: Slide 53 视觉化翻译案例: encoder-decoder 架构让不同语言和长度分布都能学习。

情境	为什么 encoder-decoder 曾有效
传统翻译	输入/输出分属不同语言, cross attention 可捕捉对齐;
学术 finetune	长输入 + 短输出, 使 decoder-only 过早生成;
长指令/对话	随着输出增长, decoder-only 更容易扩展与部署。

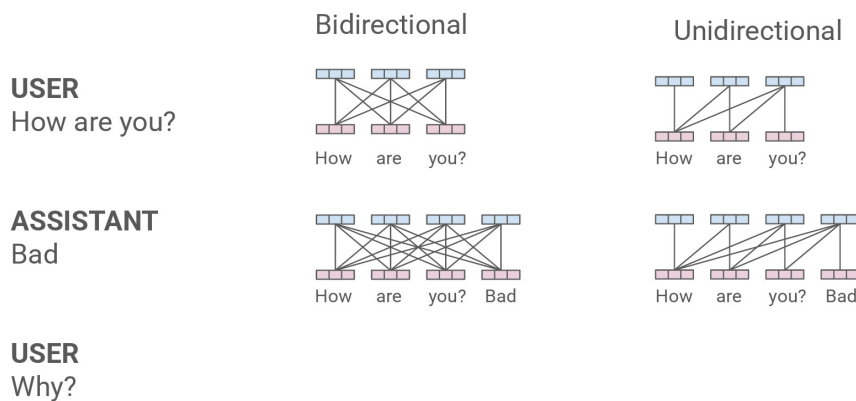
数据分布与结构选择

选择架构之前, 要先看数据的 length distribution、input vs output semantics。旧的翻译场景强调 “有界输入 + 短输出”, 而新的长对话强调 “输出也是长序列”, 容易让 decoder-only 更省成本。

5.2 多轮对话与 attention 模式

Slides 64-65 直接对比 bidirectional 与 unidirectional attention 在 multi-turn conversation 中的工程成本。双向模式对每轮都要 re-encode, 全历史参与, 而 causal 的 unidirectional 可复用之前的 hidden states, 延迟与计算负担更小。

Input attention pattern for multi-turn conversation



66

图 11: Slide 65 展示 multi-turn attention 模式, highlight 常见部署痛点。

多轮对话中的工程负担

对话系统的部署必须在 latency 与 accuracy 之间权衡; unidirectional attention 在每轮仅处理新增 message, 延迟更低、可扩展性更强。

5.3 工程 checklist

从 Slide 61 的“信息层级”讨论出发, 工程上必须整理哪些层的 activation 对哪类任务更关键。我们建议写出表格标注: 低层激活负责局部纹理/语法, 高层激活负责句子意义。再在部署时评估 cross-layer attention 是否必要。

层级信息 vs 任务类型

CV、NLP 中均存在低中高层差异; 当任务需要细粒度推理时, 需额外捕捉低层激活; 而通用生成任务更依赖 high-level semantics。

5.4 指令微调数据的长度分布

Slides 057-058 给出不同 academic dataset 的长度分布: input 通常很长、output 很短, 这在 instruction fine-tuning 中曾是一种可控结构。但随着 dataset 变成更长的 prompt-答对和 multi-turn dialogue, 这种差异变得模糊。

Additional structures in encoder-decoder compared to decoder-only

- 1. Input and target sequences are sufficiently different that using separate parameters can be effective
- 2. The target element can attend to the fully encoded representation of the input
- 3. When encoding the input sequence, all-to-all interaction among the sequence elements is preferred

59

图 12: Slide 58 显示了 academic datasets 的 length distribution，让我们理解结构选择的历史基础。

数据类型	作用
输入长、输出短	适合 encoder-decoder，encoder 提供丰富上下文；
对话/ Instruction dataset 中长输出	decoder-only 更易扩展，training pipeline 更简单；
长 prompt + 长输出	需要解耦 positional strategy，避免 cross attention 频繁重编码。

数据分布演进的教训

不要让当前 dataset 的形态决定永久结构；在数据演进的过程中把结构设计为可撤销的模式会更适合 long-lived 系统。

5.5 本章小结

本章通过翻译、指令微调与多轮对话的案例，明确结构与数据分布的对齐方式，并提供工程 checklist，避免未来部署时被结构锁住。

6 未来轨迹与可撤销结构

6.1 可撤销结构的哲学

Slides 66-67 再次用三步法收尾：发现 driving force、分析结构、预测未来。Hyung 把结构定义为“可以撤销、可以重构”的 unit，使团队在面对未知 tasks 还能快速 pivot。

Conclusion

Identify the dominant driving force as exponentially cheaper compute and associated scaling

Analyzed additional structure of encoder-decoder from the perspective of scaling

Hopefully this perspective and analysis can be useful for understanding what is happening today and predict the future trajectory

67

图 13: Slide 66 用三步法重申历史视角对未来预测的路径。

可撤销结构的定义

在 architecture design 中保留“回退通道”，避免将结构 hard-coded 到 deployment pipeline，便于 future scaling 或任务迁移。

6.2 未来预测与工程弹性

Hyung 提出：随着算力继续指数增长，我们要优先保留 general method，并在真正需要时引入 local inductive bias。这代表组织在做 research roadmap 时要准备“回退门槛”，确保每项新增结构都可在未来迅速 remove。

未来路线图的工程弹性

- 让 architecture design 支持“add & rollback”；
- 记录每一个 inductive bias 的初衷与 withdrawal 条件；
- 在 experiments 中同时追踪 scale response 与 structure cost。

6.3 工程要点清单

在 Slide 066 的三步法之外，我们还需要一个实际的工程清单：哪些指标需要定期检查、哪些结构需要可撤销的 fallback。下面的表格提供了几个值得监控的维度。

维度	检查事项
结构成本	记录每个 bias 的 parameter count、memory use 与 training instability;
Scale response	在每次算力翻倍后测量模型性能与 loss 曲线, 判断是否重新引入 structure;
Deployment metrics	延迟、吞吐、cost per token, 为结构选型提供量化依据。

持续估量的核心信念

Scale 不是一次性的实验, 而是一个持续的监控流程; 保持成对的 metric (performance & cost) 才有助于防止结构的“隐藏债务”。

6.4 本章小结

本章将未来预测与可撤销结构捆绑, 强调在 scale 的浪潮中保留“回退选项”, 以便在新的 driving force 出现时迅速调整。

7 观察与行动

7.1 关键幻灯片速读

为了方便快速回顾，下面这张表把几组代表性的幻灯片与对应的 action 关联起来，便于 future reference。

幻灯片	关注点
Slide 003	识别-理解-预测的三步法，提醒我们先看 compute，再看 structure；
Slides 038-045	结构演化的实验步骤，记录每次共享/撤销的条件；
Slide 058	instruction dataset 的长度分布，检验当前架构是否仍适配；
Slides 064-066	attention 模式与未来可撤销结构的策略，确保每次扩张都有 fallback。

观察不能停留在表面

看完幻灯片后不要只是“记住标题”，而是把每张图对应的 action 写成 checklist：算力翻倍后重跑？结构口子能撤回？否则很容易复刻“学完就忘”的模式。

7.2 实践反馈循环

每一次实验之后都要进行复盘：记录（1）该架构在当前 scale 下的 behavior，（2）指令或对话 dataset 中是否有新的偏移，以及（3）是否需要把结构 rollback。这个循环要和近期每周的 engineering review 同步，形成闭环。

反馈循环模板

- 记录 pre-scale 的 metric；
- scale 后对比 loss/latency/training stability；
- 如果 structure cost 升高且效益不突出，就回退；
- 每次 rollback 都要写入 change log，供 future team 复用。

7.3 本章小结

本章整理出多个幻灯片与工程动作的 mapping，并强调持续反馈与 rollback 的全流程复盘，为长期维护笔记中的观点提供执行层面的支撑。

8 总结与延伸

8.1 核心总结表

维度	核心洞察	实践启示
动力识别	算力指数增长是 dominant driver，短期结构不能绑住未来。	设计实验与研究指标时要 align compute growth。
架构演化	Encoder-decoder 到 decoder-only 的演进展示了需要兼顾参数共享与 cross attention。	先去除冗余结构，再在 scale 下评估是否需要添加 bias。
案例与部署	结构是否适配取决于数据分布（长度、语义）与 latency 约束。	写 checklist、画 attention 模式图，并持续跟踪部署成本。
未来策略	可撤销结构 + scale 是长期路径，必须保留跳出当前投资的能力。	每次新增结构时都记录“撤回触发条件”。

8.2 进一步阅读

- The Bitter Lesson, Rich Sutton：从历史看“少假设、多规模”的长期胜者。
- Scaling Laws for Neural Language Models, Kaplan et al.：长期 compute 与 loss 的准则。
- Transformer, Vaswani et al.：架构演进的起点与 cross attention 原理。
- Scaling Instruction-Finetuned Language Models, Chung et al. (2022)：本讲引用的长指令 fine-tune 实验。
- Hidden Technical Debt in Machine Learning Systems, Sculley et al.：结构在生产所引发的维修成本。

8.3 操作建议

落地操作梳理

- 每周为 scaling experiment 写一个 CSS (Compute-Structure-Safety) 报告：记录算力、结构变化与安全检查；
- 用幻灯片索引 (Slide 038, 065, 066) 作为 review checklist，确认是否按计划撤销结构或扩张；
- 把 tokenization/position pipeline 做成独立模块，方便 future fallback；
- 定期更新 engineering log (包括 rollback 条件与 scale reaction)，形成跨团队共享的知识库。

行动的节奏感

Hyung 提醒我们需要“做快节奏的实验 + 保持慢节奏的复盘”。快速迭代在 high compute era 中是常态，在每轮之后都要停下来问：有什么结构正在慢慢生锈？这个节奏感可以帮助团队避开“看起来有效但越来越难扩张”的路径。

8.4 本章小结

通过本讲构建的三步法，我们用历史数据标定动力、用结构拆解架构、再用 engineer mindset 设计可撤销的未来，以便面对下一个规模化浪潮。